

Smart Hospital & Resource Allocation System - Project Report

Course Code: CSC 1012 - Individual Assignment

Repository URL:

<https://github.com/shireepriyanka31-svg/Smart-Hospital-Resource-Allocation-System>

Submission Date: September 20th, 2026

Submission Student ID: AS20250645

1. Cover Page

- **Project Title:** Smart Hospital & Resource Allocation System in C
- **Course Name:** CSC 1012 - C Programming Assignment
- **Submission Date:** 20th September 2026

2. Project Overview

The Smart Hospital & Resource Allocation System is a menu-driven console application written in the C programming language. It simulates the daily patient intake, bed allocation, emergency triage sorting, and medical billing processes of a hospital. The system was implemented using parallel arrays rather than structs, in line with the assignment's guidance that struct usage is optional and parallel arrays are the primary expected approach for this module. The project was developed using the Code::Blocks IDE, with all program logic contained in a single source file, main.c.

3. High-Level Architecture

The program follows a simple modular structure: a main menu loop calls dedicated functions for each feature, and each function is responsible for a single, well-defined task (registration, billing calculation, display, sorting, reporting, or file handling). This keeps each function short and easy to test in isolation, and mirrors the lecture material's emphasis on breaking a large problem into smaller custom functions.

3.1 Program Flow

- Program starts, initializes all beds to “available”, and loads any previously saved bed status from `beds_status.txt`.
- The main menu is displayed in a loop, offering: Register Patient, Display Bed Occupancy, Display Patients by Priority, Generate Summary Report, and Save & Exit.
- Registering a patient collects input (validated), runs the billing formulas, updates the relevant bed and queue counters, prints an itemized bill, and appends a record to `patient_records.txt`.
- On exit, the current bed occupancy grid is written back to `beds_status.txt` so the state persists between runs.

3.2 List of Implemented Functions

Function	Purpose
<code>main()</code>	Program entry point; initializes beds and starts the menu loop
<code>showMenu()</code>	Displays the main menu and routes the user's choice to the correct function.
<code>readIntInRange()</code>	Reusable input helper that validates an integer falls within a required range, re-prompting on invalid input.
<code>formatMoney()</code>	Formats a float amount into a comma-separated currency string (e.g. 56750.5 to "56,750.50").

initBeds()	Sets every bed in every ward to available (0) at program start.
findFreeBed()	Searches a given ward for the first available bed and returns its index, or -1 if full.
registerPatient()	Collects patient details, allocates a bed if requested, runs all billing calculations, and stores the new patient's data.
calcWaitTime()	Calculates estimated waiting time from the current queue count for tsen specialty he cho
calcSurcharge()	Calculates the emergency surcharge based on urgency level.
calcWardCost()	Calculates total ward stay cost from days admitted and the ward's daily rate.
calcDiscount()	Calculates the 15% age-based subsidy discount for patients under 5 or over 65.
displayBill()	Prints a single patient's itemized bill in the required format
displayBedOccupancy()	Displays how many beds are occupied/free in each ward
sortAndDisplayByPriority()	Sorts registered patients by urgency level (Critical first) using bubble sort, preserving registration order for ties.
generateReport()	Produces the summary analytics report: patient counts by urgency, total revenue, total discounts, bed occupancy percentages, and the highest-paying patient.
saveBedsStatus() / loadBedsStatus()	Writes/reads the bed occupancy grid to/from beds_status.txt so bed state survives across program runs.
savePatientRecord()	Appends a permanent billing log line for each registered patient to patient_records.txt

3. Data Structure Design

This project uses parallel arrays exclusively (no structs). Every piece of data belonging to “patient i” is stored at index i across several separate arrays, so all arrays describing patients are always the same length and indexed the same way

3.1 Lookup Data (fixed at compile time)

- specialtyID[4], specialtyName[4], baseFee[4], consultTime[4], dailyCap[4] – the four doctor specialties and their fixed properties.
- queueCount[4] – how many patients currently waiting per specialty; incremented on each registration and used to calculate waiting time.
- wardIDArr[4], wardName[4], wardRate[4], wardCapacity[4] – the four hospital wards and their fixed properties.

3.2 Bed Occupancy Matrix

int bedOccupancy[4][20] – a 2D array where the first index is the ward and the second is the bed number within that ward. A value of 0 means the bed is free; 1 means it is occupied. This directly satisfies the assignment's Requirement 1 specification for bed tracking.

3.3 Patient Records (Parallel Arrays)

- patientName[100], patientAge[100], urgencyLevel[100] – core patient identity and triage data.
- specialtyIndex[100], isAdmitted[100], wardIndex[100], bedNumber[100], daysAdmitted[100] – specialty/ward assignment data.
- waitTime[100], surcharge[100], wardCost[100], grossTotal[100], discount[100], finalAmount[100] – all billing calculation results, stored so they can be reused later in the priority display and the summary report without recalculating.

MAX_PATIENTS is set to 100, so every patient-related array is sized 100 and a running patientCount variable tracks how many of those slots are currently in use.

4. GitHub Repository

The complete source code, along with the full commit history showing incremental development, is hosted publicly at:

<https://github.com/shireepriyanka31-svg/Smart-Hospital-Resource-Allocation-System>

5. Assumptions Made During Development

- If a patient requests admission but the chosen ward has no free beds, the system automatically registers them as an Outpatient (Days Admitted = 0) instead of blocking registration entirely.
- The age-based subsidy discount is applied strictly for age < 5 or age > 65, matching the boundary wording given in the assignment brief.
- Patient IDs are generated sequentially as PAT-1001, PAT-1002, etc., based on registration order within a single program run.
- Days Admitted is restricted to a range of 1–365 days as a reasonable real-world upper bound; Outpatients are automatically set to 0 days.
- Estimated waiting time is calculated once, at the moment of registration, using the specialty's queue count at that instant; it is not recalculated retroactively as other patients are seen.
- All monetary values are assumed to be in Sri Lankan Rupees (LKR) and are rounded to two decimal places for display.
- Input validation re-prompts the user for a corrected value rather than terminating the program, since a hospital system realistically cannot afford to crash on a single bad keystroke.
- Bed occupancy state is intended to persist between separate runs of the program (via `beds_status.txt`), while patient billing logs are intended to accumulate indefinitely across runs (via `patient_records.txt`, opened in append mode).